

Programming 1: Lab 5 : Conditions and Loops

1. **Street Lamps on a Highway.** The lamp posts on a highway are numbered from X to Y. Every post whose number is divisible by N is out of order. Display the numbers of the posts that are still working, along with how many of them there are. Consider the posts numbered $X \leq i < Y$.

Input: Three integers X, Y and N, each on a separate line.

Output: In the first line, the numbers of the working posts separated by a single space. In the second line, the count of working posts.

Constraints: $1 \leq X < Y \leq 10000$, $1 \leq N \leq 100$.

Sample Input	Sample Output
10 20 3	10 11 13 14 16 17 19 7
1 6 1	0

In the second case every post is divisible by 1, so no post is working and the first line of the output stays empty.

2. Take a positive integer as input and display the product of its digits. The number can be of any length.
3. **Canteen Billing Counter.** The canteen cashier enters the amount of every bill of the day one by one. To close the counter, the cashier enters -999. A bill is called a *large bill* if its amount is greater than N, otherwise it is a *small bill*. Count the large bills and the small bills of the day.

Input: The first line contains the integer N. Every following line contains one bill amount. The input ends with the value -999, which is not a bill.

Output: Print the count of large bills in the first line and the count of small bills in the second line.

Constraints: $1 \leq N \leq 100000$, at most 100 bills are entered, every bill amount is a positive integer.

Sample Input	Sample Output
100 150 80 100 260 -999	2 2

150 and 260 are greater than 100, while 80 and 100 are not.

4. Take a positive integer N and a positive integer P as input and find the value of N raised to the power P using a while loop. Do not use the `**` operator or the `pow()` function. Handle invalid cases as well.
5. Take two positive integers as input. Check whether the second number is the exact reverse of the first one. Do not use any inbuilt reverse functions.

6. Display the first N terms of the sequence of powers of 2. Do not use the `**` operator or the `pow()` function.
1, 2, 4, 8, 16, 32, till N terms
7. Take an integer as input and check whether it is a perfect number or not (a perfect number is equal to the sum of all its divisors other than itself, for example $28 = 1 + 2 + 4 + 7 + 14$). Handle invalid conditions and make your code efficient by minimizing the number of loop iterations.
8. Take a positive integer N as input and display the following pattern of N lines:

```
*  
* *  
* * *  
* * * *
```


(till N lines)
9. Take a positive integer and a single digit D (0 to 9) as input. Count how many times the digit D appears in the number. Do not convert the number into a string.
10. Take a positive integer N as input and display all its divisors, one in each line, followed by the total count of divisors. Using this count, print whether N is a prime number or not.